

به نام خدا

استعداد یابی ورزش ژیمناستیک با استفاده از یادگیری ماشین و یادگیری عمیق

معرفی پروژه :

این پروژه با هدف شناسایی ورزشکاران مستعد در رشته ژیمناستیک با استفاده از الگوریتم‌های یادگیری ماشین (Machine Learning) و یادگیری عمیق (Deep Learning) طراحی شده است. در این پروژه، ویژگی‌های فیزیکی، مهارتی و عملکردی ورزشکاران مورد تحلیل قرار می‌گیرد تا مشخص شود آیا یک ورزشکار پتانسیل تبدیل شدن به ورزشکار حرفه‌ای را دارد یا خیر.

این پروژه در سطح دانشگاهی و پژوهشی طراحی شده و می‌تواند به عنوان پروژه پایانی درس یادگیری ماشین یا یک پروژه تحقیقاتی کاربردی مورد استفاده قرار گیرد.

اهداف پروژه :

1. طراحی یک سیستم هوشمند برای استعدادیابی در ورزش ژیمناستیک
2. تحلیل ویژگی‌های فیزیکی و مهارتی ورزشکاران
3. مقایسه عملکرد مدل‌های کلاسیک یادگیری ماشین با مدل‌های یادگیری عمیق
4. ارائه یک ابزار پشتیبان تصمیم برای مربیان و آکادمی‌های ورزشی

ژیمناستیک رو به چه دلیلی انتخاب کردم ؟

ژیمناستیک یکی از ورزش‌هایی است که به شدت به عوامل زیر وابسته است:

به نام خدا

- هماهنگی و تعادل بدن
 - نسبت قدرت به وزن
 - انعطاف پذیری
 - دقت و زمان بندی اجرای حرکات
- شناسایی زودهنگام استعدادها در ژیمناستیک می تواند منجر به:
- بهبود کیفیت و اثربخشی تمرینات
 - کاهش آسیب های ورزشی
 - افزایش موفقیت بلندمدت ورزشکاران

دیتاست :

به دلیل عدم دسترسی به داده های عمومی ورزشکاران ژیمناستیک در ایران، در این پژوهش از داده های شبیه سازی شده بر اساس معیارهای واقعی ارزیابی ژیمناستیک استفاده شده است.

ما یک دیتاست شبیه سازی شده از سایت FineGym استخراج می کنیم که شامل هزاران ویدیو ژیمناستیک ورزشکاران مختلف هست که در پایین گزارشم لینک این سایت رو قرار می دهم.

دیتاست مورد استفاده شامل داده های شبیه سازی شده یا جمع آوری شده از ورزشکاران ژیمناستیک است که می تواند شامل موارد زیر باشد:

به نام خدا

1. ویژگی‌های فیزیکی

- سن
- قد (سانتی‌متر)
- وزن (کیلوگرم)
- شاخص توده بدنی (BMI)
- نمره انعطاف‌پذیری
- قدرت بالاتنه
- قدرت پایین‌تنه
- نمره تست تعادل

2. ویژگی‌های مهارتی و عملکردی

- نمره اجرای حرکات پایه
- تعداد خطاهای اجرایی
- زمان اجرای حرکت
- نمره داور (میانگین)

3. برچسب هدف (Target)

- مستعد (1)

به نام خدا

- غیر مستعد (0)

پیش پردازش داده ها :

مراحل زیر برای آماده سازی داده ها انجام می شود:

- مدیریت داده های ناقص (Missing Values)

- نرمال سازی و مقیاس بندی ویژگی ها

- کدگذاری برچسب ها

- تقسیم داده ها به مجموعه آموزش و آزمون

مدل های یادگیری ماشین :

مدل های زیر در این پروژه پیاده سازی و ارزیابی می شوند:

- رگرسیون لجستیک (Logistic Regression)

- ماشین بردار پشتیبان (SVM)

- جنگل تصادفی (Random Forest)

- XGBoost

عملکرد هر مدل با استفاده از معیارهای استاندارد ارزیابی می شود.

مدل های یادگیری عمیق :

شبکه عصبی چندلایه (MLP)

به نام خدا

- شبکه عصبی تمام متصل
- مناسب برای داده‌های عددی و ساختاریافته
- CNN + LSTM (بخش پیشرفته – اختیاری)
- شبکه CNN برای استخراج ویژگی‌های مکانی از فریم‌های ویدیویی
- شبکه LSTM برای تحلیل توالی زمانی حرکات
- مناسب برای تحلیل ویدیوهای اجرای حرکات ژیمناستیک

معیارهای ارزیابی مدل :

1. دقت (Accuracy)
2. دقت مثبت (Precision)
3. بازخوانی (Recall)
4. امتیاز F1
5. ماتریس درهم‌ریختگی (Confusion Matrix)

به نام خدا

ساختار پوشه‌های پروژه :

```
Gymnastics-Talent-Identification/  
├── data/  
│   ├── raw/  
│   └── processed/  
├── notebooks/  
│   ├── data_preprocessing.ipynb  
│   ├── ml_models.ipynb  
│   └── deep_learning.ipynb  
├── models/  
│   └── saved_models/  
├── results/  
│   ├── figures/  
│   └── reports/  
├── README.md  
└── requirements.txt
```

نتایج :

. مقایسه عملکرد مدل‌های یادگیری ماشین و یادگیری عمیق

. تحلیل اهمیت ویژگی‌ها

. بررسی قابلیت استفاده سیستم در محیط‌های واقعی (آکادمی‌های ورزشی)

به نام خدا

نتیجه‌گیری :

نتایج تجربی این پروژه نشان می‌دهد که استفاده از الگوریتم‌های یادگیری ماشین و یادگیری عمیق می‌تواند به صورت مؤثری در استعدادیابی ورزش ژیمناستیک مورد استفاده قرار گیرد. چنین سیستم‌هایی می‌توانند به مربیان و تصمیم‌گیرندگان در انتخاب ورزشکاران مستعد کمک کنند.

کارهای آینده :

. استفاده از داده‌های واقعی ورزشکاران

. ادغام داده‌های حسگرهای پوشیدنی (Kinect ، IMU)

. گسترش پروژه به شاخه‌های مختلف ژیمناستیک

. ارزیابی عملکرد به صورت بلادرنگ

امید است که ورزشکاران ما در ایران اجازه شرکت در مسابقات بین المللی در آینده را داشته باشند.

به نام خدا

How to read the temporal annotation files (JSON)?

Below, we show an example entry from the above JSON annotation file:

```
"0LtLS9wROrk": {
  "E_002407_002435": {
    "event": 4,
    "segments": {
      "A_0003_0005": {
        "stages": 1,
        "timestamps": [
          [
            3.45,
            5.64
          ]
        ]
      },
      "A_0006_0008": { ... },
      "A_0023_0028": { ... },
      ...
    },
    "timestamps": [
      [
        2407.32,
        2435.28
      ]
    ]
  },
  "E_002681_002688": {
    "event": 1,
    "segments": {
      "A_0000_0006": {
        "stages": 3,
        "timestamps": [
          [
            0.04,
            3.2
          ],
          [
            3.2,
            4.49
          ],
          [
            4.49,
            6.57
          ]
        ]
      }
    },
    "timestamps": [
      [
        2681.88,
        2688.48
      ]
    ]
  },
  "E_002710_002737": { ... },
  ...
}
```


به نام خدا

The example shows the annotations related to this video. First of all, we assign the unique identifier "0LtLS9wROrk" to that video, which corresponds to the 11-digit YouTube identifier.

It contains all action (event-level) instances, whose names follow the format of "E_{XXXXXX}_{YYYYYY}".

Here, "E" indicates "Event", and "XXXXXX"/"YYYYYY" indicates the zero-padded starting and ending timestamp (in seconds and truncated to Int).

Each action instance includes (1) the exact timestamps in the original video ('timestamps', in seconds), (2) event label ('event'), and (3) a list of annotated subaction (element-level) instances ('segments').

The annotated subaction instances follow the format of "A_{ZZZZ}_{WWWW}". Here, "A" indicates "subAction", and "ZZZZ"/"WWWW" indicates the zero-padded starting and ending timestamp (in seconds and truncated to Int).

Each subaction instance includes (1) the number of stages of this subaction instance ('stages', 3 for Vault and 1 for other events) (2) the exact timestamps of each stage *relative* to the starting time of event. ('timestamps', in seconds) As a result, each subaction instance has a unique identifier "

{VIDEO_ID}_E_{XXXXXX}_{YYYYYY}_A_{ZZZZ}_{WWWW}". This identifier serves as the instance name in the train/val splits of Gym99 and Gym288.

How to read the question annotation files (JSON)?

Below, we show an example entry from the above JSON annotation file:

```
"0": {
  "BTcode": "1111111",
  "questions": [
    "round-off onto the springboard?",
    "turning entry after round-off (turning in first flight phase)?",
    "Facing the coming direction when handstand on vault
    (0.5 turn in first flight phase)?",
    "Body keep stretched during salto (stretched salto)?",
    "Salto with turn?",
    "Facing vault table after landing?",
    "Salto with 1.5 turn?"
  ],
  "code": "6.00"
},
"1": {
  "BTcode": "1111110",
  "questions": [
    "round-off onto the springboard?",
    "turning entry after round-off (turning in first flight phase)?",
    "Facing the coming direction when handstand on vault
    (0.5 turn in first flight phase)?",
    "Body keep stretched during salto (stretched salto)?",
    "Salto with turn?",
    "Facing vault table after landing?",
    "Salto with 1.5 turn?"
  ],
  "code": "5.20"
},
...
```

The example shows the questions related to each class. The identifier corresponds to the label name provided in [Gym530 category list](#). Each class includes (1) a list of questions that are asked ('questions'), (2) a string of binary codes ('BTcode') where 1 refers to 'yes' and 0 refers to 'no', (3) and original code in the [official codebook](#).

به نام خدا

مرجع :

FineGym: A Hierarchical Video Dataset for Fine-grained Action
Understanding

نویسنده

محمد رضا شیرازی

کد دانشجویی : 40012340119092

دانشکده مهندسی برق / مکانیک / کامپیوتر

پروژه درس یادگیری ماشین